

Choosing the exit map

How the encoder decides, how the decoder decides, and what separates them

1. The decision

The decoder is cut into a ladder: a tile can leave the trunk at any of K exits, and the shallower it leaves the less it costs and the worse it looks. A 1080p frame at 256 px tiles is 40 tiles and four reachable exits, so a frame is one choice out of 4^{40} — about 10^{24} . Something has to make that choice, once per frame, and the whole method is worth exactly as much as that something is good.

Write $D(t,k)$ for the distortion tile t incurs if it leaves at exit k , and c_k for what exit k costs in units of one released decode. Then the allocation that minimises total distortion under a compute budget is the solution of a Lagrangian, and it decouples per tile:

$$k^*(t) = \arg \min_k [D(t,k) + \lambda \cdot c_k]$$

One λ for the whole frame; raise it and every tile becomes more willing to leave early. The budget is met by bisecting λ . This is the same construction a codec uses to hit a rate target, with compute in place of bits.

So the decision is not the hard part. **Getting $D(t,k)$ is.** And there the two sides of the codec are not symmetric at all.

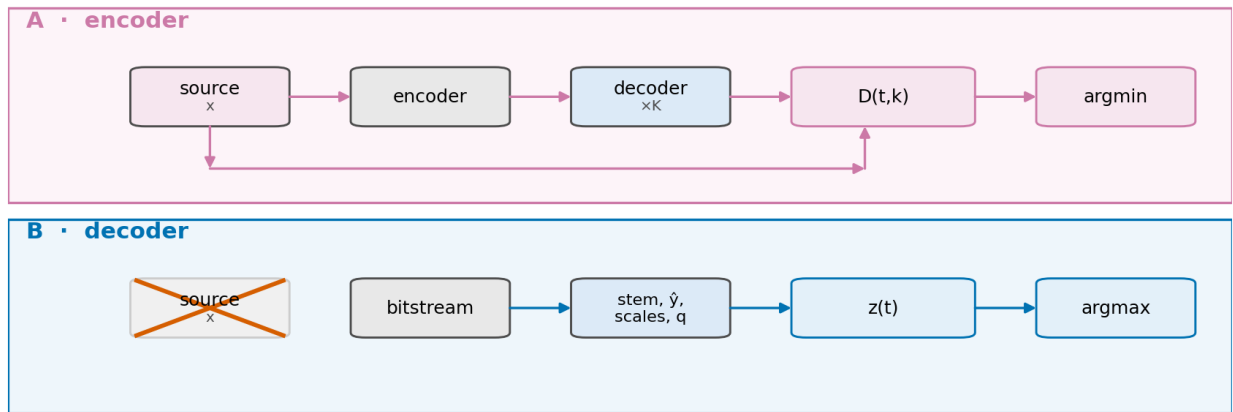


Figure 1. The asymmetry the whole design turns on. $D(t,k)$ is an error against the source. The encoder holds the source, so it can decode all K exits and measure D directly. The decoder never sees the source: for it, D is not hard to estimate, it is *absent from the input*, and no architecture recovers it. The two configurations below are the two honest responses to that.

2. Configuration A — the encoder searches

The encoder has x . It runs the decoder K times over the frame, one per exit, and reads off $D(t,k)$ exactly. No estimation, no model, no training: the table is measured. Then it bisects λ until the frame lands on the quality budget and transmits the resulting map.

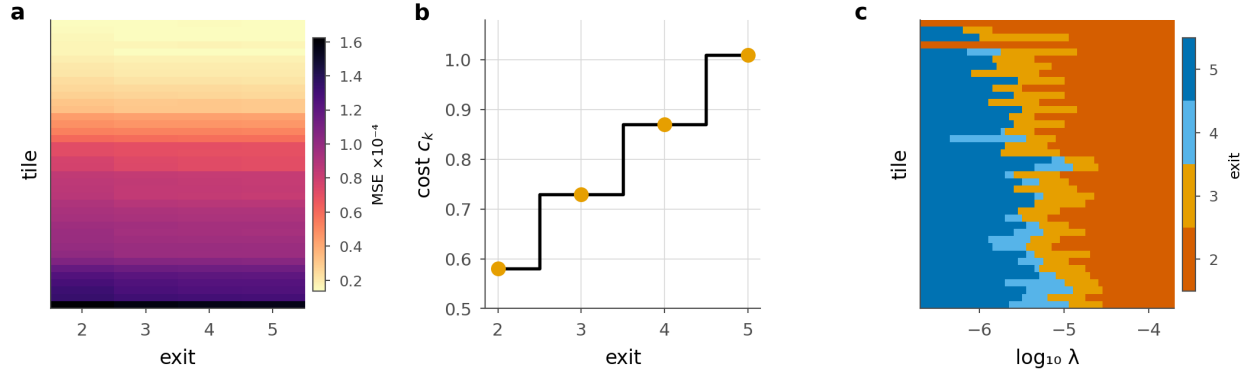


Figure 2. The search, on one real frame (Bosphorus, q32, 40 tiles). **a**, the measured table $D(t,k)$, tiles sorted by difficulty. The dominant variation is down the rows, not across them: tiles differ from each other far more than exits differ within a tile, which is why allocating per tile is worth anything at all. **b**, the cost ladder — four reachable rungs, roughly 0.14 apart. **c**, the argmin at every λ in the sweep. Tiles migrate from the deepest exit to the shallowest as λ grows, and they migrate in order of difficulty: the easy tiles at the top leave first, the hard ones at the bottom hold on longest. Above $\lambda \approx 5 \times 10^{-5}$ everything has left and the ladder is saturated.

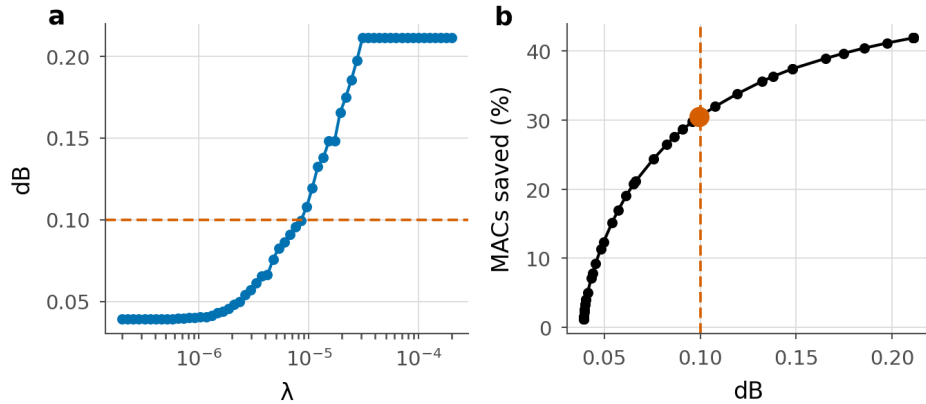


Figure 3. What the sweep traces. **a**, distortion against λ ; the dashed line is the 0.1 dB budget, and bisection finds where the curve crosses it. **b**, the same allocations as a frontier of compute against quality, with the budget point marked. The frontier is concave and ends flat: past saturation, spending more quality buys no more compute.

What it costs

Bits. Four symbols over 40 tiles is 80 bits raw, and the distribution is far from uniform, so an arithmetic coder with a per-frame histogram spends about 80 bits — 0.019% of a typical 1080p bitrate. Every bpp we report includes it.

Encoder compute. About 1.21 decodes per frame: one shared trunk pass plus the adapter and head at each exit. Encoders are already asymmetric with decoders by a large factor, and this is a search a real encoder would fold into its existing mode decision.

Decoder compute. None. It is told.

Bitstream syntax. One new field, so both ends must agree. This is the conventional design rather than the exotic one: HEVC and VVC signal block partitioning, prediction mode and the transform tree rather than having the decoder infer them.

3. Configuration B — the decoder predicts

The other honest response is to send nothing and let the decoder guess. The file is then byte-identical to a stock stream, no syntax changes, and a decoder vendor can deploy it alone. What the decoder has to work with is everything it has already

decoded before the trunk runs: the full-frame stem feature, the latent $\mathbf{z}$, the entropy model's scales, and the quality index q .

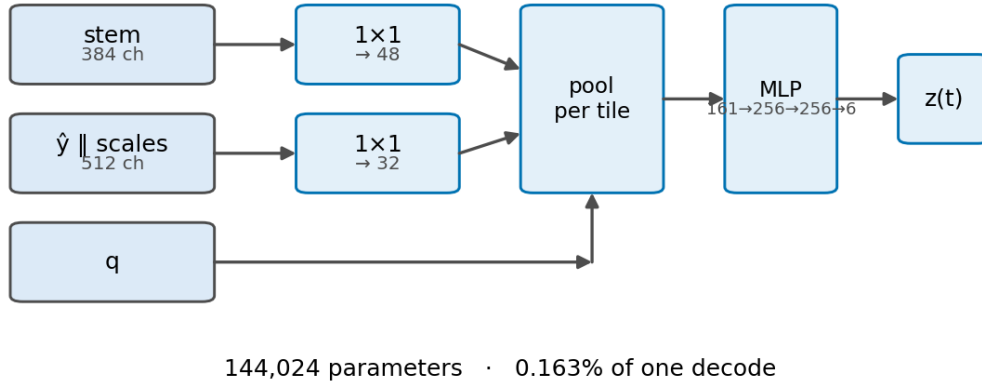


Figure 4. The predictor. Two 1×1 projections reduce the stem and the latent side, both are pooled to one vector per tile, the quality index is appended, and a three-layer MLP emits one logit per exit. Everything is per tile and pointwise, so the head adds no receptive field and no seam.

The decision mirrors A's, with the log-probability standing in for negative distortion and β for λ :

$$k_{\mathbf{z}}(t) = \arg \max_k [\log \text{softmax}(z_t)_k - \beta \cdot c_k]$$

β is bisected exactly as λ is, so the budget is met the same way. The head is 144,024 parameters and 0.163% of one decode, and that 0.163% is charged against every saving configuration B reports.

How it is trained, and how it fails

Against a **frozen** decoder, with a cost-sensitive cross-entropy to the oracle's choice: each tile weighted by the regret of choosing wrongly, so capacity goes where the decision matters and the ties are left alone. Plus loss-free load balancing — a per-exit bias nudged by usage rather than by a gradient — biased toward the *oracle's* exit distribution rather than toward uniform. Uniform is what a mixture-of-experts wants because its experts are interchangeable; ours are not, and at a high λ the oracle genuinely does send every tile to one exit.

Trained jointly with a decoder that is still moving, it can collapse. We have three routers and three outcomes, all measured: one jointly trained router collapsed to a rate-dependent constant with zero agreement with the oracle; a second jointly trained router did not collapse; the frozen-decoder recipe reaches 0.85–0.92 held-out agreement. So joint training is not automatically fatal, but it is not reliable either, and the frozen recipe is what we report.

4. What separates them

q	Allocation	Δ PSNR (dB)	MACs saved (%)
0	uniform, exit 2†	0.139	39.1
	uniform, exit 3	0.061	24.2
	uniform, exit 4	0.041	13.0
	uniform, exit 5	0.026	-1.0
	random (matched mix)	0.114	34.5
	rate-ranked (free)	0.104	34.5
	oracle	0.100	34.5
16	uniform, exit 2†	0.172	39.1
	uniform, exit 3	0.080	24.2
	uniform, exit 4	0.055	13.0
	uniform, exit 5	0.034	-1.0
	random (matched mix)	0.123	30.8
	rate-ranked (free)	0.104	30.8
	oracle	0.100	30.8
32	uniform, exit 2†	0.218	39.1
	uniform, exit 3†	0.105	24.2
	uniform, exit 4	0.067	13.0
	uniform, exit 5	0.042	-1.0
	random (matched mix)	0.134	26.7
	rate-ranked (free)	0.105	26.7
	oracle	0.100	26.7
48	uniform, exit 2†	0.253	39.1
	uniform, exit 3†	0.122	24.2
	uniform, exit 4	0.074	13.0
	uniform, exit 5	0.048	-1.0
	random (matched mix)	0.142	25.3
	rate-ranked (free)	0.107	25.3
	oracle	0.100	25.3
63	uniform, exit 2†	0.288	39.1
	uniform, exit 3†	0.138	24.2
	uniform, exit 4	0.081	13.0
	uniform, exit 5	0.054	-1.0
	random (matched mix)	0.147	24.1
	rate-ranked (free)	0.111	24.1
	oracle	0.100	24.1

Table 1. Allocations at matched compute, 0.1 dB. The two shuffled rows are the useful controls: both take the oracle's own exit histogram — the same mix of depths, hence the same average cost — and differ only in which tile gets which depth.

Given the oracle's histogram but assigned at random, quality collapses. What the allocation buys is not that some tiles run shallower; it is knowing *which* ones can.

The rate-ranked row is the one that should temper any enthusiasm for a learned router. It keeps that histogram and orders it by a signal the decoder already holds and did not have to learn: the number of bits the entropy model spent on each tile. A tile that cost more bits carries more detail and should run deeper. No parameters, no training, and the number is available before the trunk starts. At q63 a random assignment costs 0.147 dB where the oracle costs 0.100 dB at identical compute; rate-ranking costs 0.111 dB — **77% of the oracle's advantage over chance**, at 0.68–0.79 agreement with the oracle's map. Any learned router has to beat that before it has earned its 0.163%.

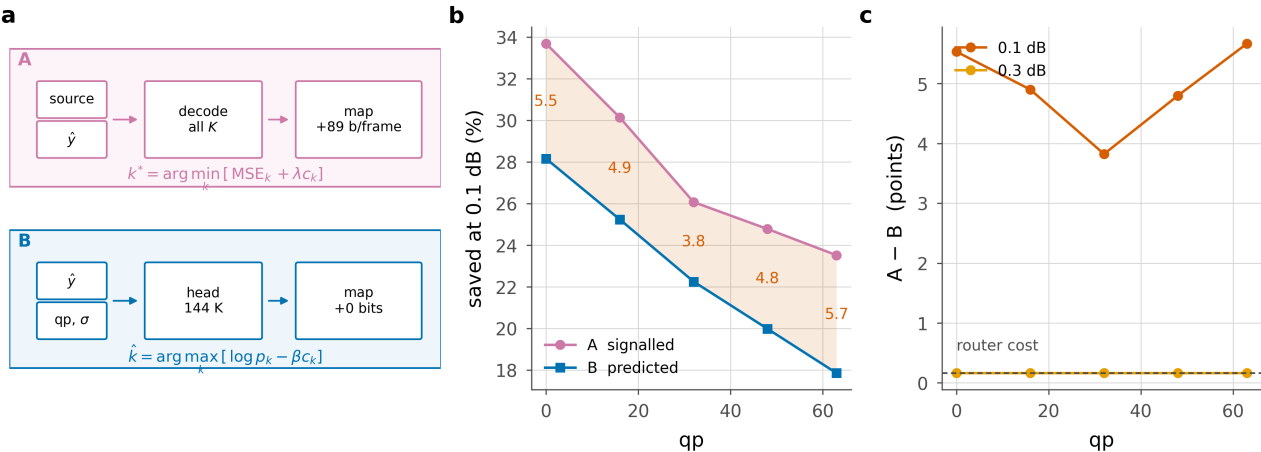


Figure 5. A against B across rate and budget. The gap is real when the budget is tight and vanishes when it is loose: once the budget saturates the ladder, both configurations send every tile to the same rung and the only difference left is the predictor's own compute.

5. Which one to use

	A — signalled	B — predicted
Optimality	exact, it is the oracle	0.55–0.86 agreement
Bits added	≈80/frame (0.019%)	zero
Decoder compute	none	0.163% of a decode
Encoder compute	≈1.21 decodes	none
File	coded payload identical, plus a field	byte-identical
Syntax	needs a new field	unchanged
Deployable by	both ends together	a decoder vendor alone
Failure mode	none observed	collapses if trained on a moving decoder

They answer different questions. A is what the system can do when the allocation is right, and it is the number to quote as the method's capability. B is what survives when nothing may change in the bitstream, and it is the number to quote as the method's deployability. Reporting only one of them would be reporting half the result.